Reglas de Integridad

en Bases de Datos

Caso de estudio: Sistema de Torneos de Videojuegos

Bases de Datos · Ingeniería en Sistemas · UTN

¿Qué pasa si no aplicamos reglas de integridad?

Datos inconsistentes, inválidos o huérfanos generan errores en reportes, cálculos y decisiones.

JUGADORES

id_jugador	nickname	email
1	xKiller	xk@mail.com
1	ShadowX	sx@mail.com
null	DragonZ	dz@mail.com
3	xKiller	kk@mail.com

→ Integridad de Entidad

PARTIDAS

id_partida	eq_local	eq_visit
10	1	2
11	99	3
12	2	null
13	5	5

→ Integridad Referencial

RESULTADOS

Id_resultado	Id_partida	ranking
1	10	8500
2	12	-200
3	13	null
4	11	15000

→ Integridad de Dominio

Estos errores existen porque la BD no tiene restricciones declaradas.

Solución → aplicar Reglas de Integridad en el diseño de la base de datos: el propio motor las hace cumplir automáticamente en cada operación.

¿Qué es la Integridad de Datos?

Conjunto de reglas que garantizan que la información almacenada sea:

precisa · consistente · válida · confiable

La integridad busca prevenir:



Errores humanos



Datos inconsistentes



Corrupción lógica



Referencias inválidas

Regla vs Restricción

Concepto	Significado	Ejemplo
Regla de integridad	Definición conceptual	`"Ranking debe ser positivo"`
Restricción de integridad	Implementación en SQL / SGBD	`CHECK (ranking >= 0)`

Tipos de Reglas de Integridad

01



Integridad de Entidad

Cada fila debe identificarse de forma única. Basada en la clave primaria.

02



Integridad Referencial

Los valores de clave foránea deben apuntar a filas existentes en la tabla padre.

03



Integridad de Dominio

Cada valor debe pertenecer al dominio válido definido para su columna.

04



Integridad de Usuario

Reglas de negocio específicas que el negocio impone más allá de los estándares.



Integridad de Entidad

01

Garantiza que cada fila de una tabla pueda ser identificada de forma única e inequívoca.
Establece dos condiciones:

① Unicidad

El valor de la PK debe ser ÚNICO para cada fila de la tabla. No pueden existir dos filas con el mismo valor.

② No Nulidad

El valor de la PK no puede ser NULL. Cada fila DEBE tener identificador definido.

Ejemplo – tabla JUGADORES:

❌ **Sin integridad**

id_jugador	nickname
1	xKiller
1	ShadowX
null	DragonZ

✅ **Con integridad**

id_jugador	nickname
1	xKiller
2	ShadowX
3	DragonZ

Se implementa en SQL con PRIMARY KEY (incluye NOT NULL + UNIQUE automáticamente). Puede ser simple (una columna) o compuesta (varias columnas).

SQL – PRIMARY KEY · Tabla paises

Primera tabla del torneo. Crea la base de datos y define países de origen de jugadores.

```sql
-- Crear la base de datos
CREATE DATABASE torneo_gaming;


-- Tabla paises: primera tabla, más simple
CREATE TABLE paises (
    id_pais     SERIAL       PRIMARY KEY,
    nombre      VARCHAR(100) NOT NULL,
    codigo_iso  CHAR(3)      NOT NULL
);
```

SERIAL

Autoincremental: PostgreSQL crea una secuencia automática. id_pais se asigna solo en cada INSERT.

PRIMARY KEY

Combina NOT NULL + UNIQUE en una sola declaración. Solo puede existir una PK por tabla.

CHAR(3) vs VARCHAR

CHAR(3) → longitud fija (p.ej. 'ARG'). VARCHAR(n) → longitud variable hasta n caracteres.

SQL – PRIMARY KEY · Formas de definición

Al crear la tabla

```sql
CREATE TABLE jugadores (
    id_jugador SERIAL PRIMARY KEY,
    nickname   VARCHAR(50) NOT NULL
);
```

PK compuesta – tabla intermedia

```sql
CREATE TABLE equipos_jugadores (
    id_equipo  INT NOT NULL,
    id_jugador INT NOT NULL,
    PRIMARY KEY (id_equipo, id_jugador)
);
```

ALTER TABLE – agregar PK a tabla existente (forma recomendada: con nombre)

```sql
-- Buena práctica: siempre nombrar las restricciones
ALTER TABLE equipos_jugadores
    ADD CONSTRAINT pk_equipos_jugadores
    PRIMARY KEY (id_equipo, id_jugador);
```

Nombrar restricciones (CONSTRAINT pk_...) permite hacer DROP CONSTRAINT para modificarlas o eliminarlas en el futuro sin recrear la tabla.

SQL – Restricción UNIQUE

Integridad de Entidad

Garantiza valores únicos en una columna sin que sea la clave primaria. Una tabla puede tener múltiples UNIQUE.

```sql
CREATE TABLE jugadores (
    id_jugador SERIAL       PRIMARY KEY,
    nickname   VARCHAR(50)  NOT NULL,
    email      VARCHAR(100) NOT NULL,
    id_pais    INT,
    ranking    INT          DEFAULT 1000,

    CONSTRAINT uq_jugadores_nickname UNIQUE (nickname),
    CONSTRAINT uq_jugadores_email    UNIQUE (email)
);
```

PRIMARY KEY vs UNIQUE

	PRIMARY KEY	UNIQUE
Por tabla	Solo 1	Múltiples
Admite NULL	✕ No	☑ Sí (uno)
Identifica fila	☑ Sí	Solo unicidad

Permite NULL

A diferencia de PRIMARY KEY, UNIQUE sí admite un valor NULL (en la mayoría de los motores). Útil para campos opcionales que deben ser únicos si se completan.

Integridad Referencial

Garantiza que las relaciones entre tablas sean válidas. Se basa en la clave foránea (FOREIGN KEY).

Regla central: No puede existir una referencia a una fila que no existe en la tabla padre.

Condición 1

La FK debe ser igual a algún valor de la PK en la tabla referenciada, o bien ser NULL si lo permite el negocio.

Condición 2

La FK puede ser NULL si la relación es opcional (ej: jugador sin país asignado aún).

Diagrama de relaciones en torneo_gaming:



✕ Violación típica: insertar en PARTIDAS un id_equipo_local = 99, siendo que no existe ningún equipo con id_equipo = 99 en la tabla EQUIPOS.

SQL – Restricción FOREIGN KEY

Agregamos FK a jugadores y creamos partidas con dos FK a la misma tabla EQUIPOS.

jugadores → FK a paises

```sql
CREATE TABLE jugadores (
    id_jugador SERIAL       PRIMARY KEY,
    nickname   VARCHAR(50)  NOT NULL,
    email      VARCHAR(100) NOT NULL,
    id_pais    INT,
    ranking    INT          DEFAULT 1000,
    CONSTRAINT uq_jugadores_nickname UNIQUE (nickname),
    CONSTRAINT fk_jugadores_pais
        FOREIGN KEY (id_pais) REFERENCES paises(id_pais)
);
```

partidas → dos FK a EQUIPOS

```sql
CREATE TABLE partidas (
    id_partida      SERIAL PRIMARY KEY,
    id_equipo_local INT NOT NULL,
    id_equipo_visit INT NOT NULL,
    fecha_hora TIMESTAMP NOT NULL,
    CONSTRAINT fk_local
      FOREIGN KEY (id_equipo_local)
      REFERENCES equipos(id_equipo),
    CONSTRAINT fk_visit
      FOREIGN KEY (id_equipo_visit)
      REFERENCES equipos(id_equipo)
);
```

Acciones Referenciales

Qué hace el motor cuando se modifica (UPDATE) o elimina (DELETE) una fila referenciada en la tabla padre.

CASCADE

Propaga el DELETE/UPDATE a todas las filas hijas automáticamente.

```
ON DELETE CASCADE
```

RESTRICT

Bloquea la operación si existen hijas. Verificación inmediata.

```
ON DELETE RESTRICT
```

NO ACTION

Similar a RESTRICT, pero la verificación ocurre al final de la transacción.

```
ON DELETE NO ACTION
```

SET NULL

Establece NULL en la FK de las filas hijas. La columna debe admitir NULL.

```
ON DELETE SET NULL
```

SET DEFAULT

Establece el valor DEFAULT en la FK de las filas hijas.

```
ON DELETE SET DEFAULT
```

SQL – ON DELETE CASCADE · Tabla resultados

Si se elimina una partida, sus resultados se eliminan automáticamente. Garantiza consistencia.

```sql
CREATE TABLE resultados (
    id_resultado  SERIAL PRIMARY KEY,
    id_partida    INT    NOT NULL,
    puntos_local  INT    NOT NULL DEFAULT 0,
    puntos_visit  INT    NOT NULL DEFAULT 0,
    duracion_min  INT,

    -- Una partida tiene exactamente un resultado
    CONSTRAINT uq_resultado_partida UNIQUE (id_partida),

    -- Si se borra la partida, se borra el resultado
    CONSTRAINT fk_resultado_partida
        FOREIGN KEY (id_partida)
        REFERENCES partidas(id_partida)
        ON DELETE CASCADE
);
```

¿Por qué CASCADE aquí?

Un resultado sin partida no tiene sentido lógico (registro huérfano). CASCADE mantiene la consistencia automáticamente.

¿Cuándo usar RESTRICT?

Si quisiéramos proteger el historial: que no se pueda borrar una partida si ya tiene resultado cargado.

Integridad de Dominio

Garantiza que cada valor almacenado sea atómico y pertenezca al dominio válido definido para su columna.

Tipo de Dato

INT, VARCHAR(50), DATE, BOOLEAN…
Define el formato y rango nativo.

```
ranking INT
```

NOT NULL

El campo es obligatorio.
No puede quedar sin valor.

```
nickname NOT NULL
```

CHECK

Condición que debe cumplir cada valor. Puede combinar AND, OR, IN, BETWEEN.

```
CHECK (ranking >= 0)
```

DEFAULT

Valor automático cuando no se proporciona uno en el INSERT.

```
DEFAULT 1000
```

SQL – CHECK y NOT NULL · Tabla jugadores completa

Agregamos todas las restricciones de dominio sobre los datos de jugadores.

```sql
CREATE TABLE jugadores (
    id_jugador SERIAL        PRIMARY KEY,
    nickname   VARCHAR(50)   NOT NULL,
    email      VARCHAR(100)  NOT NULL,
    fecha_nac  DATE          NOT NULL,
    ranking    INT           DEFAULT 1000,
    id_pais    INT,

    CONSTRAINT uq_jugadores_nickname UNIQUE (nickname),
    CONSTRAINT uq_jugadores_email    UNIQUE (email),
    CONSTRAINT fk_jugadores_pais
        FOREIGN KEY (id_pais) REFERENCES paises(id_pais),
    CONSTRAINT ck_jugadores_ranking
        CHECK (ranking >= 0 AND ranking <= 9999)
);
```

NOT NULL

nickname, email y fecha_nac son obligatorios. El motor rechaza el INSERT si se omiten.

CHECK compuesto

ranking >= 0 AND ranking <= 9999 valida que el valor esté dentro del rango. Se puede combinar con OR, IN, BETWEEN.

CHECK con IN (tabla equipos_jugadores)

rol IN ('capitan','titular','suplente') — acepta exactamente esos tres valores.

SQL – Restricción DEFAULT · Tabla equipos

Especifica un valor automático cuando el INSERT no incluye esa columna.

```sql
CREATE TABLE equipos (
    id_equipo       SERIAL        PRIMARY KEY,
    nombre          VARCHAR(100)  NOT NULL,
    fecha_creacion  DATE          NOT NULL DEFAULT CURRENT_DATE,
    activo          BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_equipos_nombre UNIQUE (nombre)
);

-- INSERT usando defaults (solo se provee el nombre):
INSERT INTO equipos (nombre) VALUES ('Team Dragon');

-- Equivale exactamente a:
INSERT INTO equipos (nombre, fecha_creacion, activo)
VALUES ('Team Dragon', CURRENT_DATE, TRUE);
```

DEFAULT con función

CURRENT_DATE inserta la fecha de hoy automáticamente.
También: NOW(), CURRENT_TIMESTAMP.

DEFAULT con valor fijo

DEFAULT TRUE, DEFAULT 1000, DEFAULT 'activo' → valor constante cuando no se escribe el campo.

SQL – Restricción NOT NULL · Tabla partidas

Impide que una columna almacene valores nulos. El campo siempre debe tener un valor concreto.

```sql
CREATE TABLE partidas (
    id_partida      SERIAL       PRIMARY KEY,
    id_equipo_local INT          NOT NULL,
    id_equipo_visit INT          NOT NULL,
    fecha_hora      TIMESTAMP    NOT NULL,
    fase            VARCHAR(20)  NOT NULL,
    sede            VARCHAR(100),        -- permite NULL
    CONSTRAINT fk_local
      FOREIGN KEY (id_equipo_local) REFERENCES equipos(id_equipo),
    CONSTRAINT fk_visit
      FOREIGN KEY (id_equipo_visit) REFERENCES equipos(id_equipo)
);

-- Agregar / quitar NOT NULL con ALTER TABLE:
ALTER TABLE partidas ALTER COLUMN fase SET NOT NULL;
ALTER TABLE partidas ALTER COLUMN sede DROP NOT NULL;
```

NULL ≠ vacío ≠ cero

NULL significa «valor desconocido o no aplicable». No es 0 ni ‘ ’. Afecta comparaciones: NULL = NULL da UNKNOWN.

NOT NULL + DEFAULT

Se pueden combinar: el campo nunca es NULL y tiene valor automático si no se especifica.

Integridad Definida por el Usuario

Reglas de negocio específicas que no encajan en las categorías estándar. Van más allá del tipo de dato y CHECK simple.

CHECK avanzado – reglas de negocio en torneo_gaming

```sql
-- Un equipo no puede enfrentarse a sí mismo
CONSTRAINT ck_equipos_distintos
    CHECK (id_equipo_local <> id_equipo_visit),

-- El rol del jugador solo puede tener tres valores válidos
CONSTRAINT ck_rol_jugador
    CHECK (rol IN ('capitan', 'titular', 'suplente')),

-- Los puntos no pueden ser negativos
CONSTRAINT ck_puntos_local  CHECK (puntos_local >= 0),
CONSTRAINT ck_puntos_visit  CHECK (puntos_visit >= 0)
```

Cuando CHECK no alcanza → usar TRIGGER o Procedimiento Almacenado

TRIGGER

Función que se ejecuta automáticamente ante INSERT / UPDATE / DELETE. Puede validar reglas complejas entre múltiples tablas.

Procedimiento Almacenado

Encapsula lógica de negocio. Se invoca explícitamente. Permite transacciones complejas.

Regla de negocio

Lógica en la capa de aplicación. Más frágil: se puede saltear si se accede directamente a la BD.

Resumen – Reglas de Integridad

Tipo	Garantiza	Restricción SQL	Ejemplo en torneo_gaming
Entidad	Filas únicas e identificables	`PRIMARY KEY · UNIQUE`	`id_jugador SERIAL PK`
Referencial	Relaciones válidas entre tablas	`FOREIGN KEY + ON DELETE/UPDATE`	`id_pais → paises(id_pais)`
Dominio	Valores dentro del dominio	`NOT NULL · CHECK · DEFAULT`	`CHECK (ranking >= 0)`
De usuario	Reglas de negocio complejas	`CHECK · TRIGGER · Stored Proc.`	`equipos distintos en partida`

Esquema construido hoy:

`paises → jugadores → equipos_jugadores ← equipos → partidas → resultados`